

Strukture podataka i algoritmi

Algoritmi pretraživanja

dr Danijel Čabarkapa

Akademija strukovnih studija Šabac

Odsek za medicinske i poslovno-tehnološke studije

Sadržaj:

1. Algoritmi linearnog pretraživanja
2. Algoritmi binarnog pretraživanja

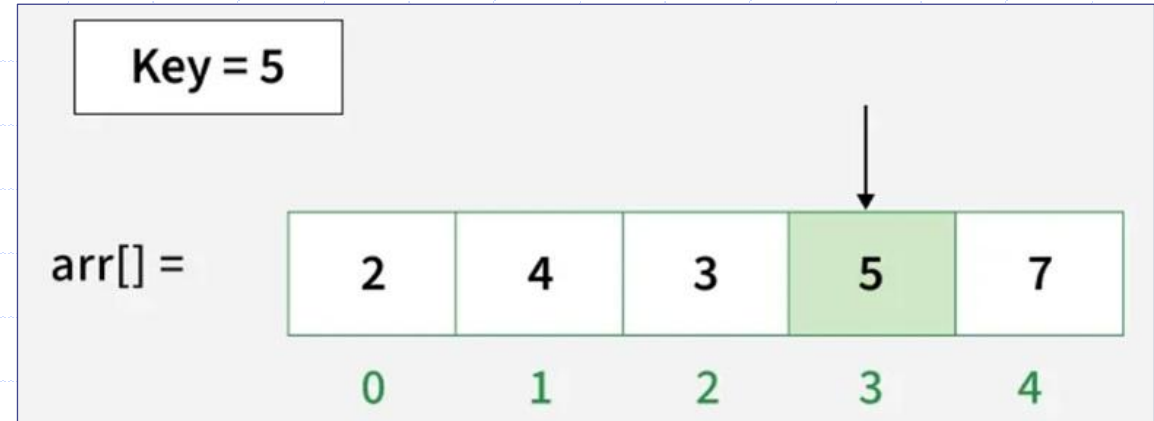
1. Algoritmi linearnog pretraživanja

Pretraživanje podataka

- Pretraživanje je proces dobijanja određenih podataka iz nizova/kolekcija na osnovu specifičnih kriterijuma
- Algoritmi za pretraživanje se koriste u bazama podataka, u aplikacijama za obradu teksta, u brojnim inženjerskim algoritmima, pretragama weba itd...
- Cilj algoritama za pretraživanje je najčešće da odrede da li u datom nizu/kolekciji postoji traženi podatak i da vrate njegovu poziciju (indeks)
- Operacija pretraživanja može se izvršiti nad različitim strukturama podataka
- Pretraživanje niza (ili liste podataka) podrazumeva pronalaženje elementa unutar niza korišćenjem ključa za pretraživanje, a ključ je jedinstvena vrednost koja se koristi za identifikaciju elemenata u nizu
- Za kolekcije složenih tipova podataka, specifična komponenta podataka mora biti identifikovana kao ključ, pa se ključ u tom slučaju može sastojati od više komponenti, što je poznato i kao složeni ključ

Algoritmi za pretraživanje podataka

- Princip pretraživanja:

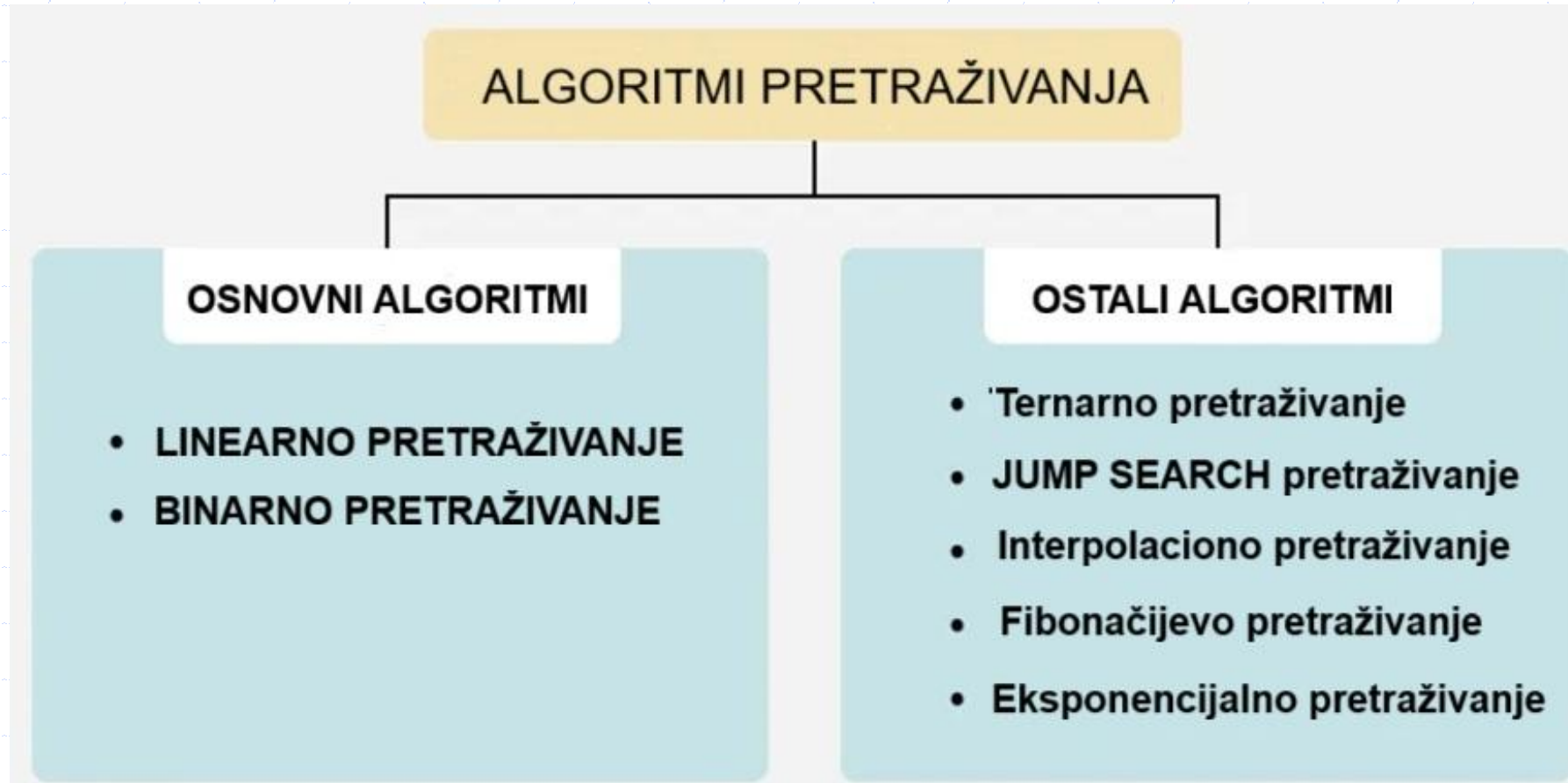


- PRIMENE:**



Algoritmi za pretraživanje podataka

- Osnovni algoritmi pretraživanja su: **LINEARNO** i **BINARNO** pretraživanje
- Svi drugi algoritmi pretraživanja su izvedeni iz osnovnih

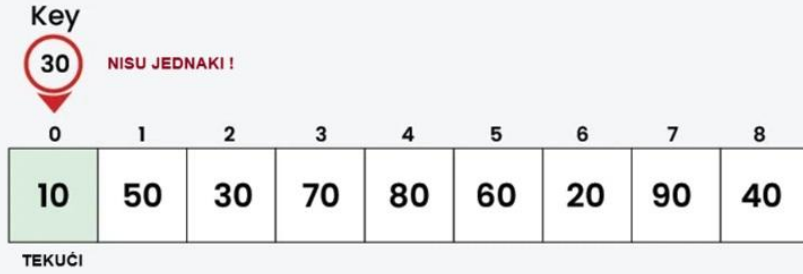


Pretraživanje podataka

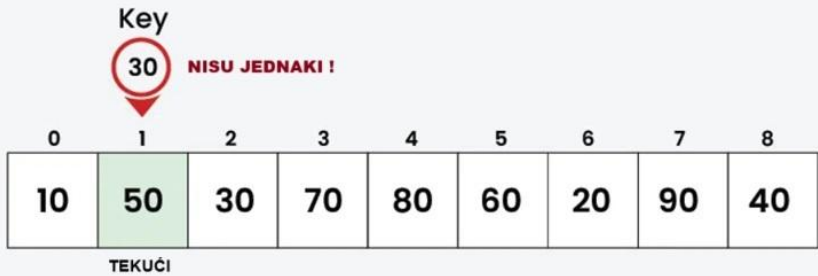
- U računarstvu se uglavnom koriste dva različita algoritma za pretraživanje:
 - 1) **brute force** algoritam
 - 2) **podeli pa vladaj** (*Divide & Conquer*) algoritam
- **Brute force** algoritam proverava svaki član sekvencijalno upisanih podataka niza. Provera elemenata niza moguća je **iterativnim** ili **rekurzivnim** pristupom
- **„Podeli pa vladaj“** algoritam polazi od pretpostavke da je niz već sortirán. Algoritam prvo posmatra srednji član niza, ako je taj član jednak broju koji se traži vraća se rezultat, a ako nije proverava se da li je taj član veći od onog koji se traži. Ako jeste, to znači da se pretraživanje može lokalizovati na “donju” polovinu niza, a u suprotnom se pretraživanje usmerava ka “gornjoj” polovini niza. Ovakvo pretraživanje se još naziva binarno pretraživanje
- Postoji još i tehnika pretraživanja **„Two Pointers“** kod koje se pretraživanje niza vrši počevši sa obe kraja preko dva pokazivača

Linearno pretraživanje

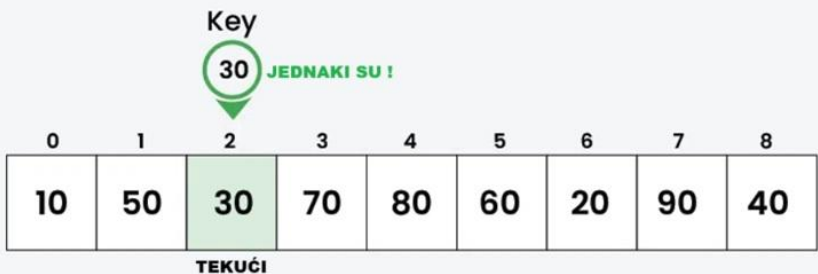
01 POREDI SE KEY SA SVAKIM ELEMENTOM NIZA POČEVŠI OD PRVOG



02 POREDI SE KEY SA DRUGIM ELEMENTOM NIZA, POŠTO NISU JEDNAKI PRELAZI SE NA SLEDEĆI ELEMENT



03 POREDI SE KEY SA TREĆIM ELEMENTOM NIZA, POŠTO SU JEDNAKI, PRETRAŽIVANJE SE PREKIDA



- Najjednostavnije rešenje problema pretraživanja niza je sekvencijalni ili linearni algoritam pretraživanja
- U primeru posmatramo **nesortirani niz brojeva** [10, 50, 30, 70, 80, 60, 20, 90, 40] i potrebno je utvrditi da li se broj 30, koji je **ključ (key)** nalazi u nizu
- U prvom koraku polazi se od prvog elementa (tekućeg) koji se upoređuje sa ključem, pošto prvi element ne sadrži vrednost ključa prelazi se na drugi element niza
- U drugom koraku se drugi element niza (50) poredi sa vrednošću ključa (30), a pošto nisu jednaki, prelazi se na treći element niza
- Proces se ponavlja sve dok se u trećem koraku ne pronađe traženi broj 30 na trećoj poziciji (**indeks $i=2$**)
- Šta ako broj 30 nije u nizu? Vršiti se iteracija kroz celu sekvencu niza, element po element niza dok se ne pronađe traženi element ili dok se svi elementi ne ispituju i dođe se do poslednjeg u nizu

Linearno pretraživanje

- U Pythonu se linearno pretraživanje može izvršiti korišćenjem operatora `in` u `if-else` ciklusu:

```
if key in Niz:
    print(„Ključ je pronađen u nizu.")
else:
    print(„Ključ nije pronađen u nizu.")
```

- Funkcija implementira algoritam sekvencijalnog pretraživanja, koja vraća logičku vrednost (`True` ili `False`) koja ukazuje na uspeh ili neuspeh pretraživanja. Iterativna petlja se koristi za prolazak kroz sekvencu niza tokom koje se svaki element upoređuje sa traženom vrednošću. Ako je element u sekvenci, petlja se prekida i vraća se vrednost `True`. U suprotnom, vrši se potpuni prolazak kroz niz i vraća se `False` nakon što se petlja završi

```
def linearSearch(vrednosti_niza, target):
    n=len(vrednosti_niza)
    for i in range(n):
        # Ako je target na i-toj poziciji, return True
        if vrednosti_niza[i]==target:
            return True
    return False    # Ako nije pronađen, return False
```

Linearno rekurzivno pretraživanje - primer

```
def linear_search(arr, target):  
    # Linearna pretraga nalaženja elementa date liste  
    # Parametri: arr(list)-lista koja se pretražuje, target-vrednost elementa liste koja se traži  
    # Returns: int-index elementa liste koji je pronađen, u suprotnom vraća -1  
  
    for i in range(len(arr)):  
        if arr[i] == target:  
            return i  
    return -1  
  
# Primer:  
arr = [2, 3, 4, 10, 40]  
target = 10  
result = linear_search(arr, target)  
if result != -1:  
    print(f"Linear Search: Element found at index {result}")  
else:  
    print("Linear Search: Element not found")
```

Output

```
Linear Search: Element found at index 3
```

Linearno rekurzivno pretraživanje - primer

```
1 import random
2
3 def linear_recursive_search(arr, target, index=0):
4     if index >= len(arr):
5         return -1 # broj nije pronađen
6     if arr[index] == target:
7         return index # broj je pronađen
8     return linear_recursive_search(arr, target, index + 1)
9
10 # Unos za podatke veličine liste i opsega
11 user_size = int(input("Unesi veličinu liste: "))
12 user_min = int(input("Unesi donju granicu opsega: "))
13 user_max = int(input("Unesi gornju granicu opsega: "))
14
15 # Generisanje random liste na osnovu ulaznih podataka
16 random_list = [random.randint(user_min, user_max) for _ in range(user_size)]
17 print("Random lista brojeva:", random_list)
18
19 # Unos vrednosti broja koji se traži
20 user_target = int(input("Unesi broj koji treba da se pronađe: "))
21
22 # Izvršavanje linearne rekurzivne pretrage
23 index_found = linear_recursive_search(random_list, user_target)
24 if index_found == -1:
25     print(f"Broj {user_target} nije pronađen u listi.")
26 else:
27     print(f"Broj {user_target} je pronađen na indeksu {index_found}.")
```

Unesi veličinu liste: 20

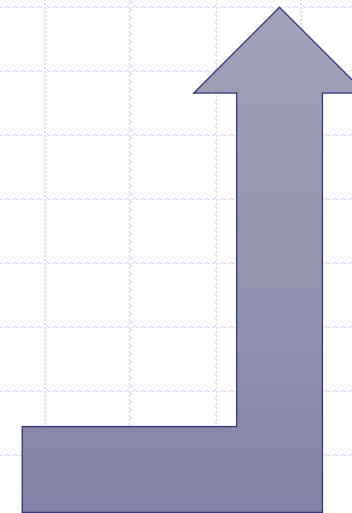
Unesi donju granicu opsega: 300

Unesi gornju granicu opsega: 1000

Random lista brojeva: [439, 385, 628, 757, 442, 405, 367, 512, 428, 327, 963, 431, 695, 607, 597, 535, 775, 861, 915, 937]

Unesi broj koji treba da se pronađe: 700

Broj 700 nije pronađen u listi.



Linearno pretraživanje - karakteristike

- Vremenska složenost linearnog algoritma pretraživanja:
 - **Najbolji slučaj** je ako je ključ na prvom indeksu i tada je složenost $O(1)$
 - **Najgori slučaj** je kada je ključ prisutan na poslednjem indeksu, tj. na suprotnoj strani od kraja sa kojeg je pretraga liste počela, pa je složenost za ovaj slučaj $O(n)$, gde je n veličina niza ili liste
 - Za prosečan slučaj složenost je $O(n)$
- Prostorna složenost je $O(1)$, jer se osim iteracije kroz listu, ne koristi nijedna druga promenljiva
- Primene linearnog algoritma pretraživanja:
 - Linearno pretraživanje se koristi za pronalaženje elemenata u nesortiranim listama
 - Linearno pretraživanje je poželjnije od binarnog za male skupove podataka (nizove)
 - Linearno pretraživanje se obično koristi za pronalaženje elemenata unutar povezanih listi, pri čemu se svaki čvor proverava sekvencijalno dok se ne pronađe željeni element
 - Linearno pretraživanje je mnogo lakše za razumevanje i implementaciju u poređenju sa binarnim, ekspopnencijalnim ili ternarnim pretraživanjem
- Prednosti i nedostaci linearnog algoritma pretraživanja: ne zahteva dodatni memorijski prostor, primenljiv za manje skupove podataka i za nizove bilo kog tipa, spor za velike skupove podataka jer je složenost $O(n)$ i nije primenljiv za relativno velike nizove

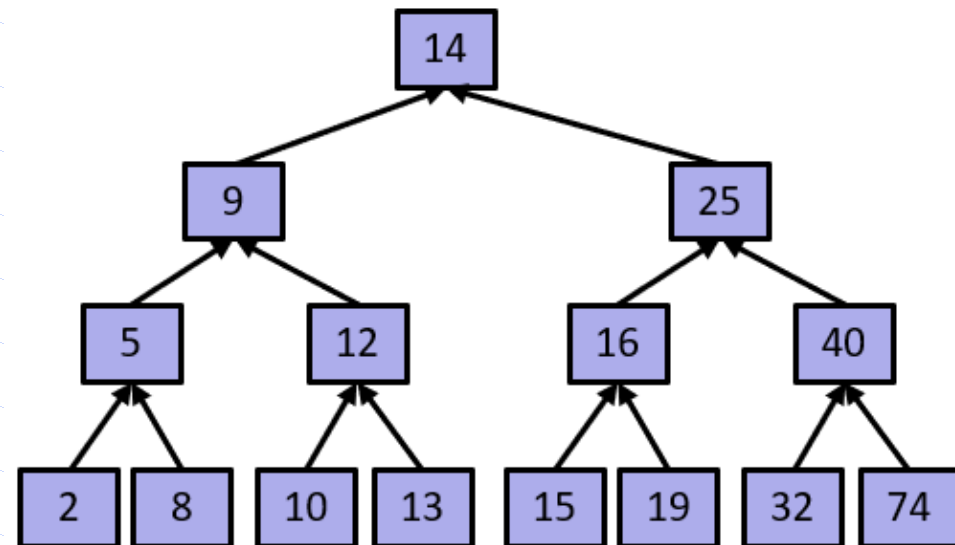
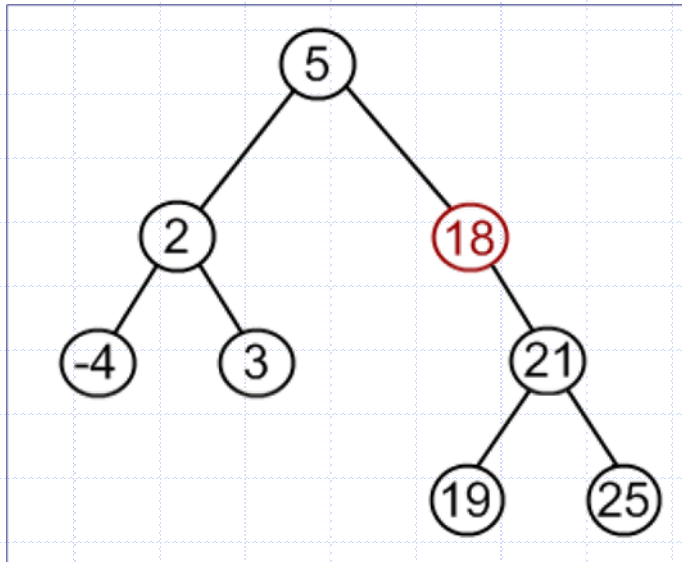
2. Algoritmi binarnog pretraživanja

Binarno pretraživanje

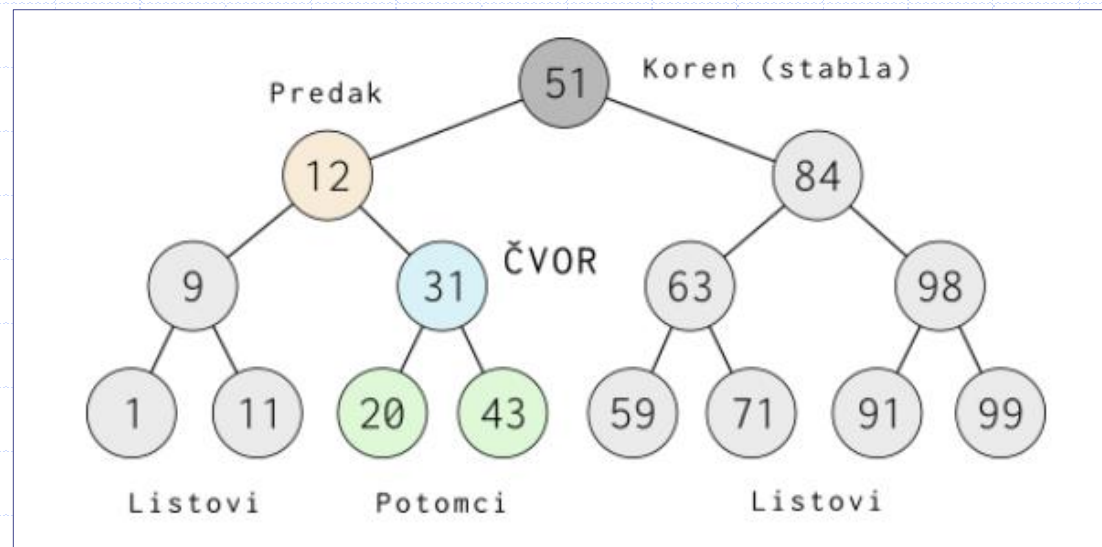
- Algoritam binarnog pretraživanja se koristi za pronalaženje elementa iz prethodno sortirane liste ili niza
- To je brz i efikasan algoritam za pretraživanje elementa, ali je njegov glavni nedostatak što je potrebna unapred sortirana lista
- Za najgori slučaj, vremenska složenost izvršavanja binarnog algoritma pretraživanja je $O(\log n)$ dok je za linearnu pretragu $O(n)$
- Princip funkcionisanja binarnog algoritma pretraživanja:
 - Pretraživanje započinje deljenjem date liste na pola, ako je traženi broj ili ključ manji od vrednosti srednjeg elementa, onda će se pretraživanje nastaviti u prvoj (levoj) polovini liste, a u suprotnom, ako je ključ veći od srednje vrednosti, onda će se pretraga lokalizovati samo u desnu polovinu liste
 - Postupak se ponavlja svaki put dok se ne pronađe element ili dok se ne proverí cela lista
 - U slučaju nenumeričke liste podataka (lista stringova), elementi liste se sortiraju po abecednom redu (slično načinu na koji se lista kontakata čuva u telefonu)

Binarna stabla pretraživanja

- **Binarna stabla pretraživanja BST** predstavljaju najznačajniju strukturu podataka koja se koristi u algoritmima pretraživanja
- BST je uređeno binarno stablo za koje važi da za svaki čvor sa vrednošću **n**, vrednosti svih čvorova iz njegovog levog podstabla su manje ili jednake **n**, a vrednosti svih čvorova iz njegovog desnog podstabla su veće od **n**
- BST stabla omogućavaju efikasniju pretragu podataka jer su podaci ispod svakog čvora podeljeni na one koji su "levo" tj. manji, i one koji su "desno" tj. veći od podatka u posmatranom čvoru



Binarna stabla pretraživanja



- Osnovni element BST strukture je broj, jedinstveni ključ ili "čvor" (node)
- Svaki čvor je povezan je sa najviše jednim čvorom sa prethodnog nivoa hijerarhije, i najviše dva čvora na sledećem nivou hijerarhije (npr. čvor #31 je jedinstveni celobrojni podatak, njegov „predak“ je čvor #12, a „potomci“ su čvorovi #20 i #43)
- Čvorovi bez potomaka (#1, #11, #20, #43, #59, #71, #91 i #99) prepoznaju se kao „listovi“
- BST pravilo: Sve vrednosti koje se nalaze u podstablu sa leve strane posmatranog čvora, moraju biti manje od vrednosti koja je pripisana datom čvoru, a sve vrednosti koje se nalaze u desnom podstablu, moraju biti veće (npr. levo podstablo čvora #12 sadrži čvorove #1, #9 i #11, a desno podstablo, čvorove #20, #31 i #43)

Binarno pretraživanje

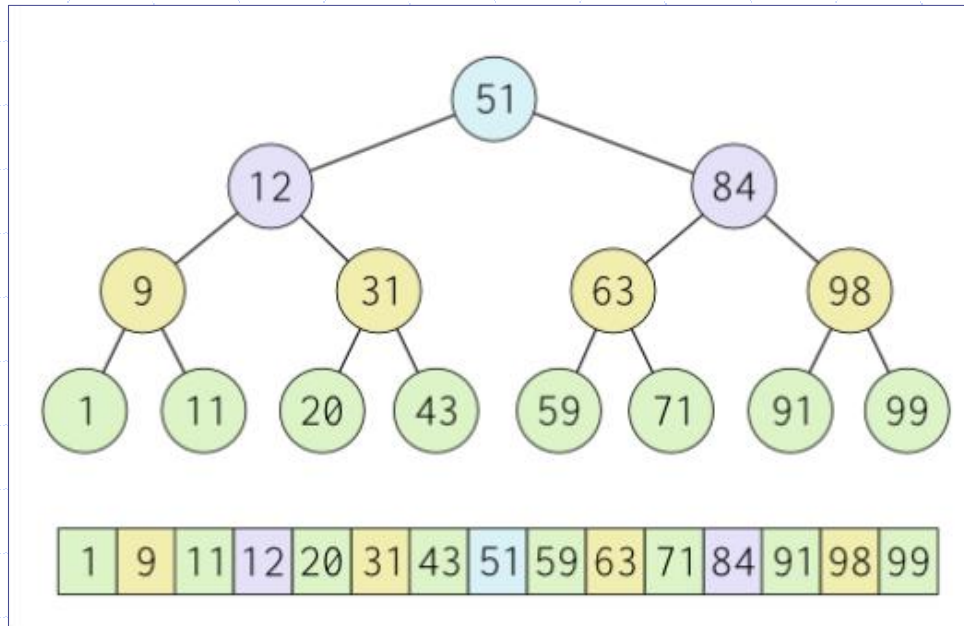
- Ukoliko je niz elemenata neuređen, kao npr: 51, 9, 1, 20, 31, 43, 98, 59, 63, 99, 84, 91, 11, 71, 12 a treba da se proveriti da li se u datom nizu nalazi broj 71, algoritam od početka ispituje svaki element niza, dok ne pronađe traženi broj, ili utvrđuje da traženi broj ne postoji u nizu (broj 71 je u nizu i može se pronaći posle najviše 14 pokušaja, ako je za obradu jednog elementa niza potrebna 1 μ s, vrednost 71 će biti pronađena za 14 μ s)
- Ukoliko niz nije ureden, nema drugih opcija i (bar u najgorem slučaju), doslovno se moraju uzeti u obzir redom svi elementi
- Ako npr. baza podataka sadrži podatke za 250 miliona osoba (brojeva), i ako je traženi broj pri kraju niza, u najgorem slučaju će traženje podatka trajati preko 4. minuta (~250s) što nije prihvatljivo
- Da li bi (i u kojoj meri) ureden niz bio od pomoći?
- **Uređeni niz:** prvobitni niz 51, 9, 1, 20, 31, 43, 98, 59, 63, 99, 84, 91, 11, 71, 12 uredimo u rastući poredak: 1, 9, 11, 12, 20, 31, 43, 51, 59, 63, 71, 84, 91, 98, 99

Binarno pretraživanje

- Ukoliko je niz elemenata neuređen, kao npr: 51, 9, 1, 20, 31, 43, 98, 59, 63, 99, 84, 91, 11, 71, 12 a treba da se proveriti da li se u datom nizu nalazi broj 71, algoritam od početka ispituje svaki element niza, dok ne pronađe traženi broj, ili utvrđuje da traženi broj ne postoji u nizu (broj 71 je u nizu i može se pronaći posle najviše 14 pokušaja, ako je za obradu jednog elementa niza potrebna 1 μ s, vrednost 71 će biti pronađena za 14 μ s)
- Ukoliko niz nije ureden, nema drugih opcija i (bar u najgorem slučaju), doslovno se moraju uzeti u obzir redom svi elementi
- Ako npr. baza podataka sadrži podatke za 250 miliona osoba (brojeva), i ako je traženi broj pri kraju niza, u najgorem slučaju će traženje podatka trajati preko 4. minuta (~250s) što nije prihvatljivo
- Da li bi (i u kojoj meri) ureden niz bio od pomoći?
- **Uređeni niz:** prvobitni niz 51, 9, 1, 20, 31, 43, 98, 59, 63, 99, 84, 91, 11, 71, 12 uredimo u rastući poredak: 1, 9, 11, 12, 20, 31, 43, 51, 59, 63, 71, 84, 91, 98, 99

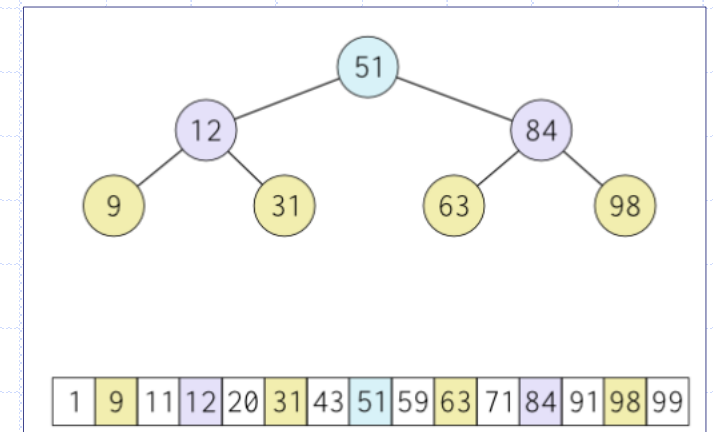
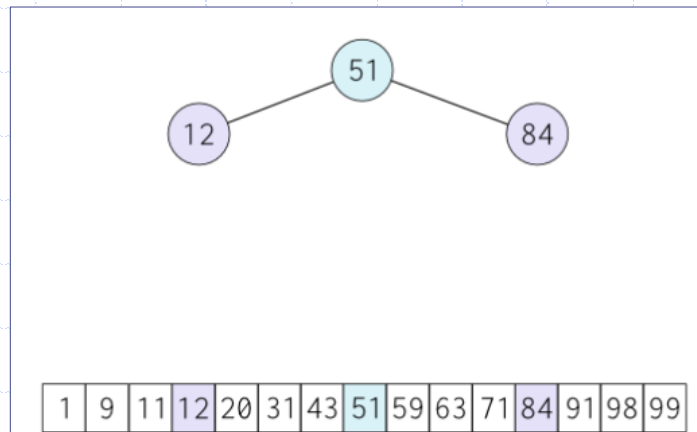
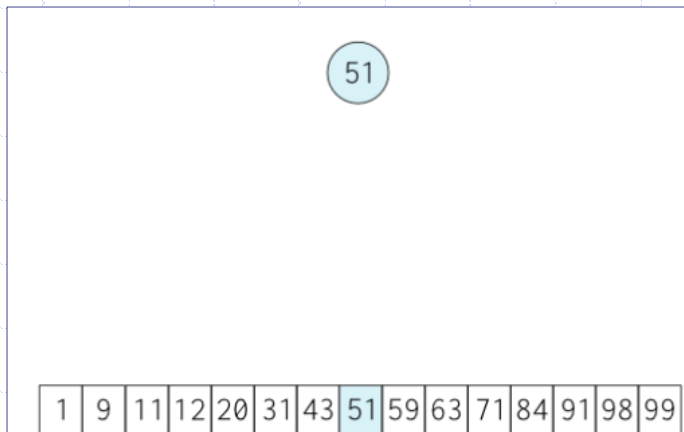
Binarno pretraživanje

- **Uređeni niz:** prvobitni niz **51, 9, 1, 20, 31, 43, 98, 59, 63, 99, 84, 91, 11, 71, 12** uredimo u rastući poredak: **1, 9, 11, 12, 20, 31, 43, 51, 59, 63, 71, 84, 91, 98, 99**
- Broj **12** se sada može pronaći iz 4 pokušaja (što jedobro), ali pretraga **71** i dalje podrazumeva prolazak kroz skoro ceo niz
- Uređenost niza u slučaju linearne pretrage pomaže samo u određenim okolnostima i onda kada se traže elementi koji su „blizu početka“ niza
- Jedno od mogućih rešenja, podrazumeva raspoređivanje elemenata niza u poredak **binarnog stabla**

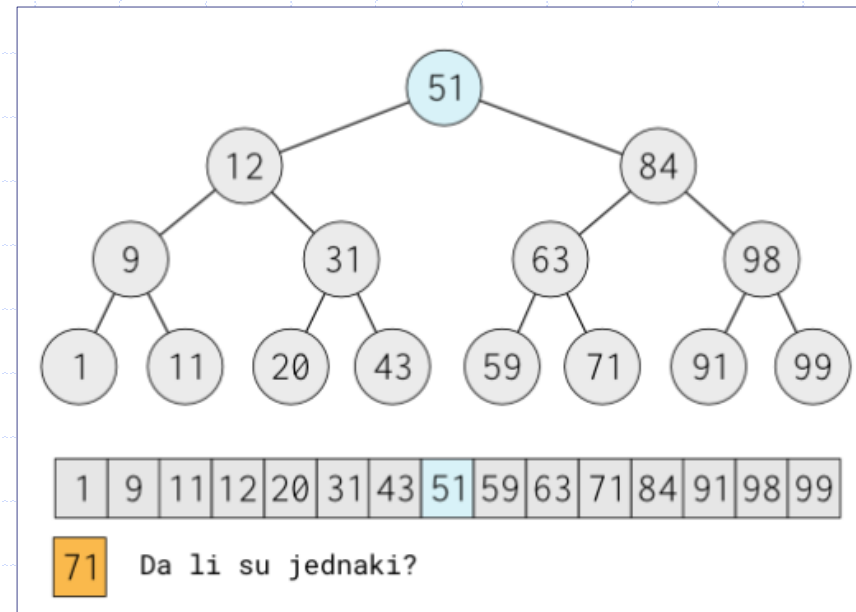
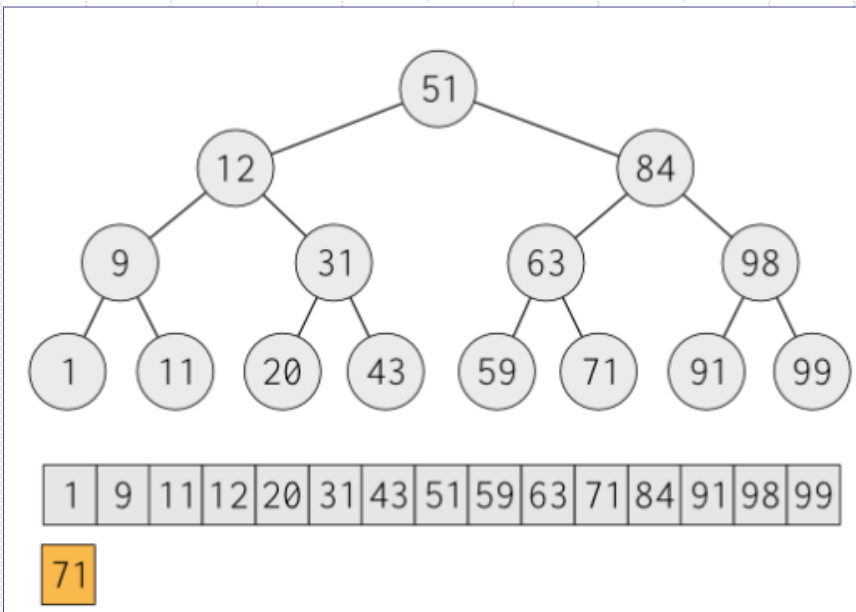


Formiranje BST stabla (priprema strukture)

- Za početak treba izabrati središnji element niza kao koren stabla (središnji element je broj 51)
- Da bi BST uslov bio zadovoljen, u svakom koraku bismo „sredine levih i desnih podnizova“ i povezujemo ih sa čvorovima koji su već uneti u stablo
- Zatim vrednosti na sredini levog i desnog odeljka prvobitnog niza (čvorovi #12 i #84) povezujemo sa korenom stabla (čvor #51)
- Potom se čvorovi #12 i #84 povezuju sa četiri čvora (#9, #31, #63, #98), koji predstavljaju sredine preostala četiri odeljka početnog niza
- Na kraju se preostali elementi (koji su sredine preostalih "osmina" niza) takođe povezuju sa stablom, po prethodno navedenim pravilima

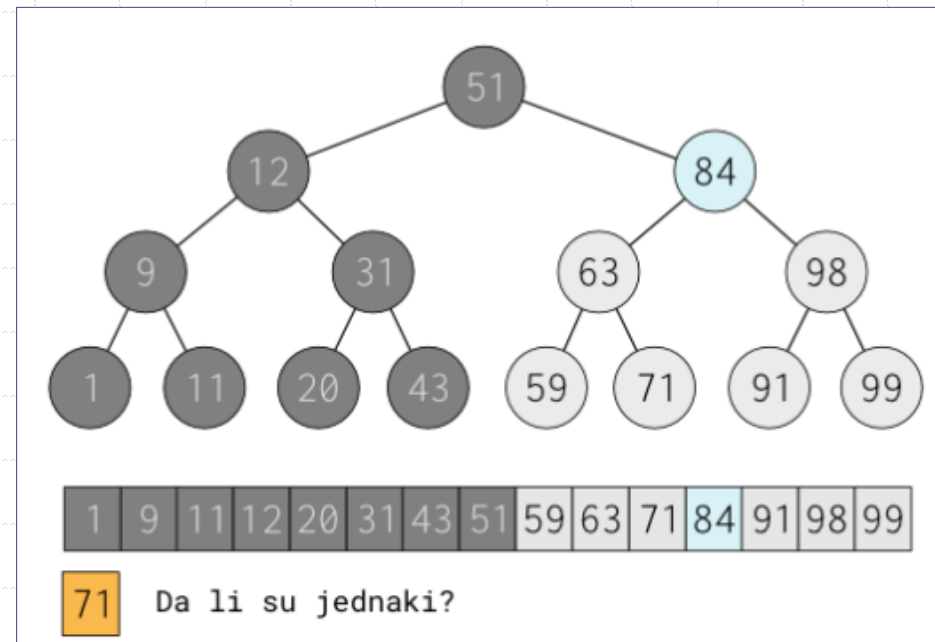
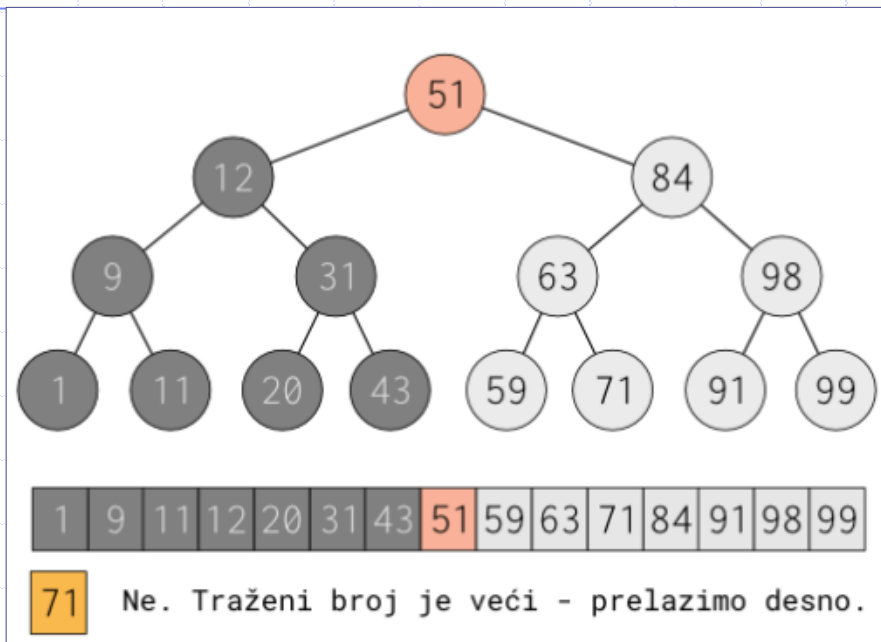


Pretraživanje BST stabla



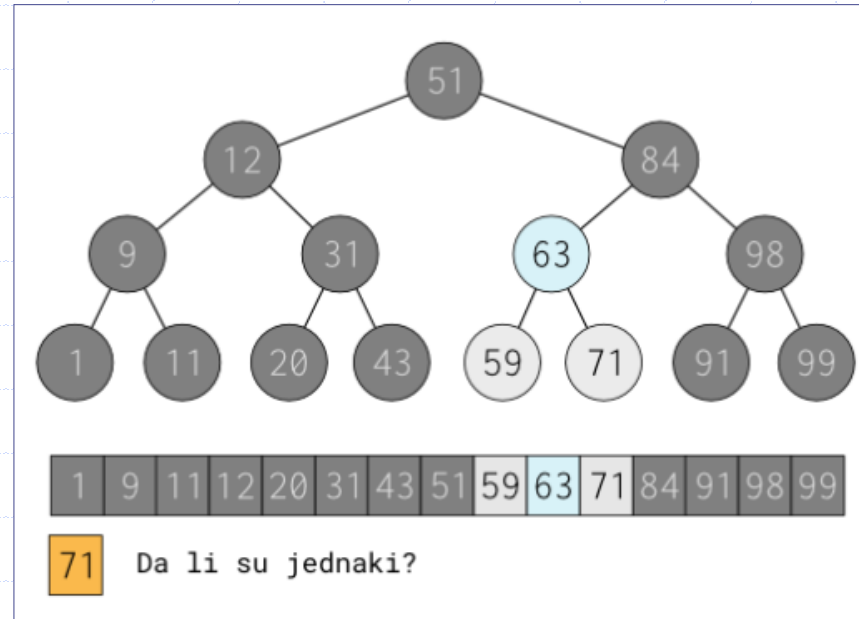
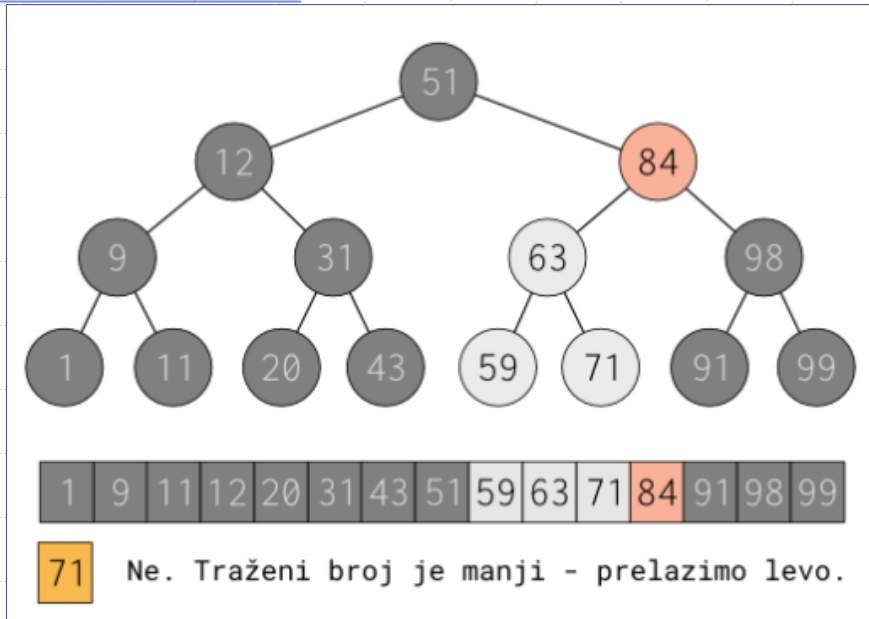
- Potrebno je pronaći broj #71 u uređenom nizu
- U prvom koraku pristupa se korenom čvoru stabla, i proverava se da li koreni čvor sadrži traženu vrednost
- Budući da u prvom koraku nije pronađena vrednost #71, i budući da koreni čvor nije jedan od "listova" stabla, pretraga se nastavlja

Pretraživanje BST stabla



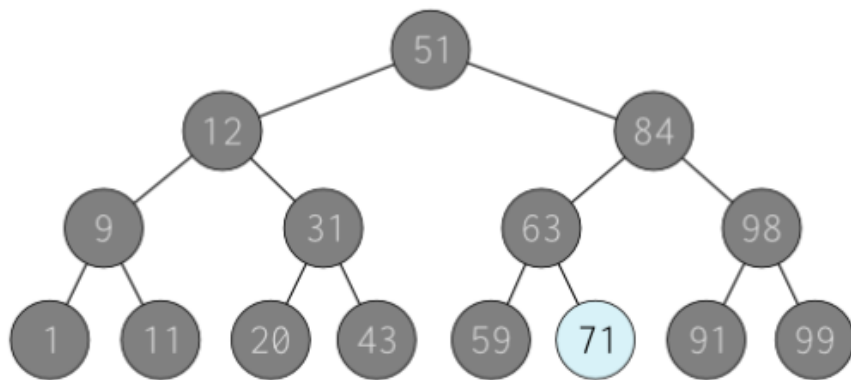
- Korenoski čvor (**#51**) je manji je od traženog broja (**#71**), pa se iz dalje pretrage može isključiti cela leva "polovina" stabla, jer su sve vrednosti u levom podstablu manje od vrednosti (**#51**)
- U sledećem koraku, ispituje se susedni desni čvor (koren desnog podstabla), i budući da traženi broj nije pronađen, pretraga se nastavlja u levom podstablu čvora **#84** (s obzirom na to da je „ključ za pretragu” **#71** manji od vrednosti čvora **#84**)

Pretraživanje BST stabla



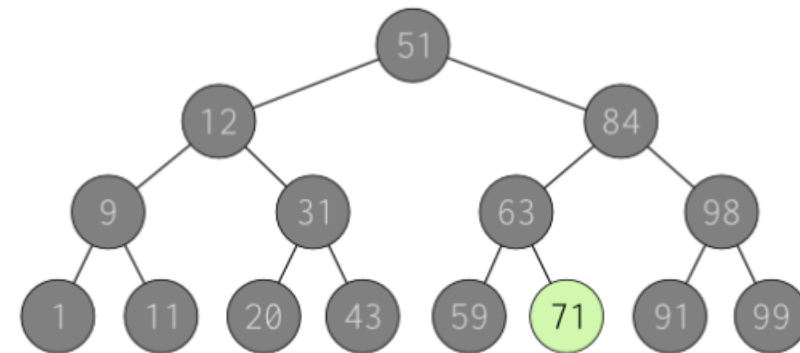
- Pre prelaska na čvor #63, iz dalje pretrage se isključuje celo desno podstablo čvora #84 (celo desno podstablo se može isključiti iz dalje pretrage u jednom koraku, pošto to podstablo sadrži vrednosti koje su veće od 84)
- Po prelasku na čvor #63, sledi novo poređenje, ponovno nema poklapanja, a budući da je tražena vrednost 71, veća od ispitivane vrednosti 63, pretraga prelazi na desno podstablo čvora #63 (i pri tom se levo podstablo isključuje iz dalje pretrage)

Pretraživanje BST stabla



1 9 11 12 20 31 43 51 59 63 71 84 91 98 99

71 Da li su jednaki?



1 9 11 12 20 31 43 51 59 63 71 84 91 98 99

71 Da. Traženi broj je uspešno pronađen.

- Na kraju se vrši još jedno poređenje i tražena vrednost je pronađena iz svega 4 pokušaja!
- Broj koraka pronalaženja određenog elementa (u pravilno balansiranom BST stablu pretrage), zavisi isključivo od visine stabla h i ne zavisi od ukupnog broja elemenata, pa je za pronalaženje elementa u BST stablu koje povezuje preko 250 miliona elemenata, u najgorem slučaju potrebno najviše 29 koraka, tj. $\sim 29\mu s$, što je ogromna razlika u odnosu na $> 4min$, što je bio rezultat u slučaju linearne pretrage

Binarno pretraživanje

- Razmatra se niz $arr[] = \{2, 5, 8, 12, 16, 23, 38, 56, 72, 91\}$ i traži se vrednost **23**

POČETAK Pronaći broj 23 binarnim pretraživanjem

arr[] =

0	1	2	3	4	5	6	7	8	9
2	5	8	12	16	23	38	56	72	91

KORAK 1 Key (23) je veći od tekućeg srednjeg elementa (16). Pretraga na desnu stranu

arr[] =

0	1	2	3	4	5	6	7	8	9
2	5	8	12	16	23	38	56	72	91

Low = 0 Mid = 4 High = 9

KORAK 3 Ako je Key jednak vrednosti srednjeg elementa, broj je nađen i prekida se pretraga

arr[] =

0	1	2	3	4	5	6	7	8	9
2	5	8	12	16	23	38	56	72	91

Low = 5 High = 6
Mid = 5

KORAK 2 Key je manji od tekućeg srednjeg elementa (56). Pretraga u levu stranu.

arr[] =

0	1	2	3	4	5	6	7	8	9
2	5	8	12	16	23	38	56	72	91

Low = 5 Mid = 7 High = 9

Rekurzivno binarno pretraživanje - kôd

Niz `arr[] = {2, 3, 4, 10, 40}` i traži se vrednost **10**

```
# Python3 iterativna binarna pretraga
# Vraća indeks x u datom nizu arr

def binarySearch(arr, low, high, x):
    while low <= high:
        mid = low + (high - low) // 2

        # Da li je x na srednjem mid?
        if arr[mid] == x:
            return mid

        # Ako je x veće, bez leve polovine
        elif arr[mid] < x:
            low = mid + 1

        # Ako je x manje, bez desne polovine
        else:
            high = mid - 1
```

```
# Ako element nije u nizu
# vraća se -1
return -1

# Driver Code
if __name__ == '__main__':
    arr = [2, 3, 4, 10, 40]
    x = 10

    # Poziv funkcije
    result = binarySearch(arr, 0, len(arr)-1, x)
    if result != -1:
        print("Element is present at index", result)
    else:
        print("Element is not present in array")
```

Output

```
Element is present at index 3
```

Iterativno binarno pretraživanje - kôd

Niz `arr[] = {2, 3, 4, 10, 40}` i traži se vrednost **10**

```
# Python3 iterativna binarna pretraga
# Vraća indeks x u datom nizu arr, ili -1
def binarySearch(arr, low, high, x):

    # Provera za osnovni slučaj
    if high >= low:
        mid = low + (high - low) // 2

        # Ako je element na sredini mid
        if arr[mid] == x:
            return mid

        # Ako je element manji od srednjeg
        # može biti samo u levom podnizu
        elif arr[mid] > x:
            return binarySearch(arr, low, mid-1, x)
```

```
# Else, element je u desnom podnizu
else:
    return binarySearch(arr, mid + 1, high, x)

# Element se ne nalazi u sekvenci niza
else:
    return -1

# Driver Code
if __name__ == '__main__':
    arr = [2, 3, 4, 10, 40]
    x = 10
    # Poziv funkcije
    result = binarySearch(arr, 0, len(arr)-1, x)
    if result != -1:
        print("Element is present at index", result)
    else:
        print("Element is not present in array")
```

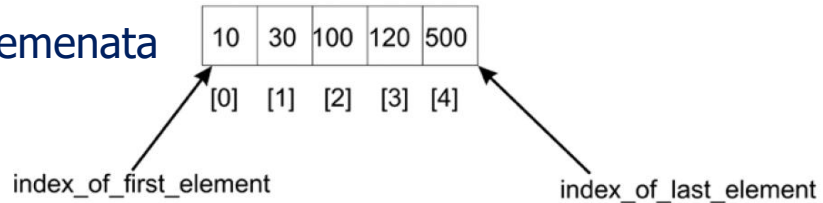
Output

```
Element is present at index 3
```

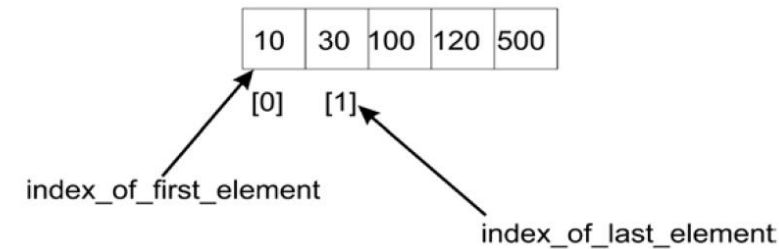
Binarno pretraživanje - primer

Niz {10, 30, 100, 120, 500} i treba pronaći poziciju broja 10

Lista od 5 elemenata



Indeks prvog i poslednjeg elementa prve polovine liste



```
def binary_search_iterative(ordered_list, term):
    size_of_list = len(ordered_list) - 1
    index_of_first_element = 0
    index_of_last_element = size_of_list
    while index_of_first_element <= index_of_last_element:
        mid_point = (index_of_first_element + index_of_last_element)/2
        if ordered_list[mid_point] == term:
            return mid_point
        if term > ordered_list[mid_point]:
            index_of_first_element = mid_point + 1
        else:
            index_of_last_element = mid_point - 1
    if index_of_first_element > index_of_last_element:
        return None
```

```
list1 = [10, 30, 100, 120, 500]
search_term = 10
index_position1 = binary_search_iterative(list1, search_term)
if index_position1 is None:
    print("The data item {} is not found".format(search_term))
else:
    print("The data item {} is found at position {}".format(search_term,
index_position1))
```

The data item 10 is found at position 0
The data item structure is found at position 3

Binarno pretraživanje

- Binarno pretraživanje podrazumeva višestruko deljenje prostora pretraživanja (tj. liste u kojoj se traženi element može nalaziti) na pola. Počinje se sa kompletnom listom i u svakoj iteraciji izračunava se srednja tačka. Nakon jednog poređenja pretraživanje se nastavlja u jednom od podnizova sa približno $n/2$ članova
- U svakoj iteraciji veličina liste se smanjuje za polovinu (npr. u I iteraciji veličina liste je n , u II iteraciji postaje $n/2$, u III iteraciji je $n/2^2$, a nakon k iteracija veličina liste postaje $n/2^k$ i u tom trenutku veličina liste će biti jednaka 1, tj. $n/2^k=1$)
- Primenom logaritma na obe strane jednačine biće:

$$\begin{aligned} \Rightarrow \log_2(n) &= \log_2(2^k) \\ \Rightarrow \log_2(n) &= k \log_2(2) \\ \Rightarrow k &= \log_2(n) \end{aligned}$$

- Na primer, za $n=1024$ u ovom algoritmu, potrebno je u najgorem slučaju 10 poređenja, dok u prosečnom slučaju kod brute force algoritma treba 512 poređenja
- Nedostatak - algoritam binarne pretrage funkcioniše samo ako je niz unapred sortiran

Ternarno pretraživanje

- Karakteristika binarnog pretraživanja je da se pri svakom koraku deli niz koji se pretražuje na dva podniza i pomoću uslova određuje u kojem podnizu se nastavlja pretraga
- **Algoritam ternarnog pretraživanja** skoro potpuno se podudara s binarnim, sa tom razlikom da se umesto jedne srednje vrednosti **mid** biraju dve i to na sledeći način:

$$\text{mid1} = l + (r-l)/3$$

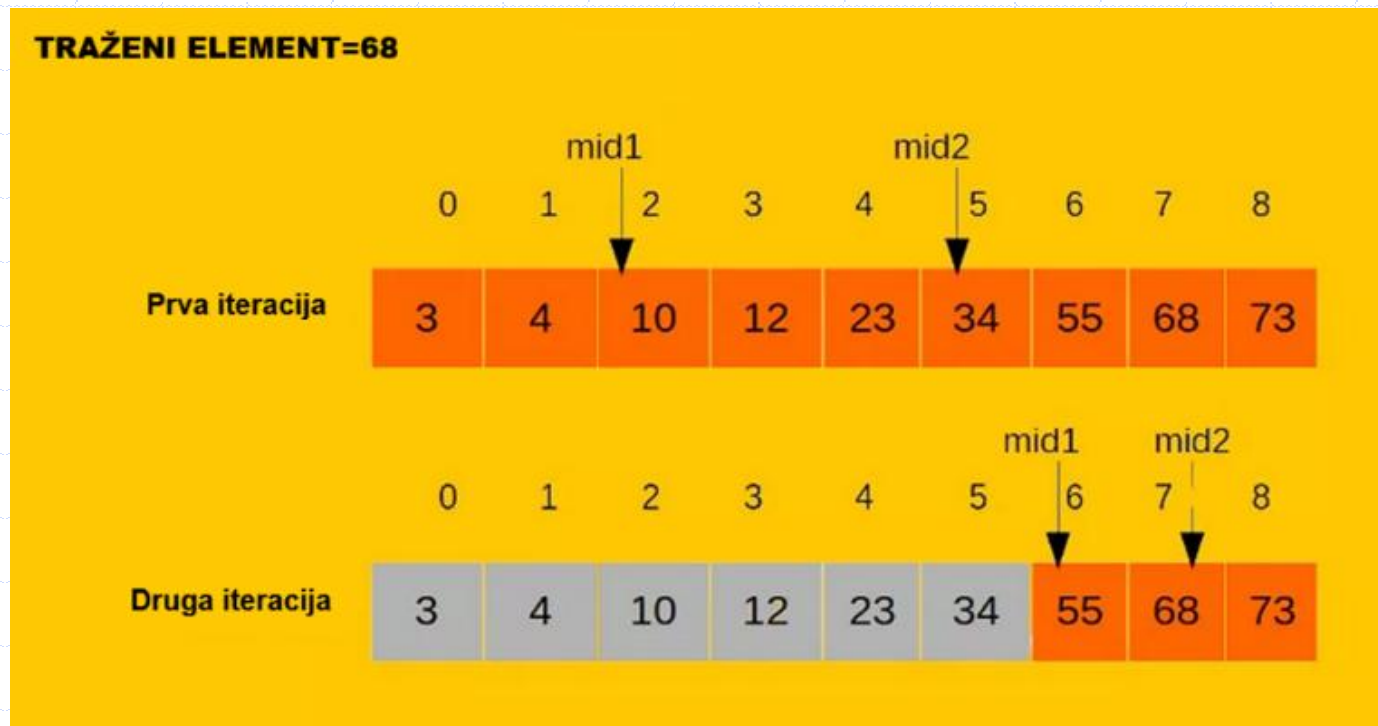
$$\text{mid2} = r - (r-l)/3$$

gde su **r** i **l** desna i leva granica trenutnog podniza

- Zatim upoređujemo traženi broj **x** sa vrednostima **array[mid1]** i **array[mid2]**. Ako je **x** različit od obe vrednosti (nismo pronašli rešenje), imamo tri slučaja:
 - 1) **x** je manji od **array[mid1]**: **x** se (ako ga ima) nalazi u intervalu **[l,mid1]**
 - 2) **x** je veći od **array[mid2]**: **x** se (ako ga ima) nalazi u intervalu **[mid2,r]**
 - 3) inače: **x** se nalazi u intervalu **[mid1,mid2]**
- Napravi li se analogija sa binarnim pretraživanjem, problem se pri svakom koraku svodi na 1/3 prethodnog problema pa je složenost algoritma **$O(\log_3(n))$**

Ternarno pretraživanje

- Ternarno pretraživanje je tipa „zavadi-pa-vladaj“ koji se koristi za pronalaženje pozicije tražene vrednosti unutar rastućeg ili opadajućeg unimodalnog niza (U-shaped ili $\cap$ -shaped)
- Element sortiranog niza se pronalazi njegovom podelom na tri dela putem dve srednje tačke (midpoints)
- Vremenska složenost ovog algoritma pretraživanja je $O(\log_3 n)$



Izbor 2 srednje tačke niza:

→ **mid1**: Pozicija elementa prve trećine tekućeg prostora pretrage

→ **mid2**: Pozicija elementa druge trećine tekućeg prostora pretrage.

U svakom koraku poredi se traženi element sa mid1 i mid2 i odlučuje se u kojem delu niza se nastavlja pretraga:

- Ako je traženi broj = mid1, vraća se njegov indeks
- Ako je traženi broj = mid2, vraća se njegov indeks
- Ako je traženi broj < mid1, pretraćuje se prva trećina niza
- Ako je traženi broj između mid1 i mid2, pretraćuje se srednja trećina
- Ako je traženi broj > mid2, pretraćuje se poslednja trećina

Ternarno pretraživanje

Ternarno pretraživanje - postupak

1. Prostor pretrage $\rightarrow$ [low, high] indeksi niza

2. Srednje tačke:

$$\text{mid1} = \text{low} + (\text{high} - \text{low}) / 3$$

$$\text{mid2} = \text{high} - (\text{high} - \text{low}) / 3$$

2. Poređenja sa mid1 i mid2:

Slučaj 1: $\text{arr}[\text{mid1}] == \text{key}$, vraća mid1 indeks

Slučaj 2: $\text{arr}[\text{mid2}] == \text{key}$, vraća mid2 indeks

Slučaj 3: $\text{key} < \text{arr}[\text{mid1}]$, traženi element je u prvoj trećini niza, ažurira se prostor pretrage u [low, mid1 - 1]

Slučaj 4: $\text{arr}[\text{mid1}] < \text{key} < \text{arr}[\text{mid2}]$, traženi element je u drugoj trećini niza, ažurira se prostor pretrage u [mid1 + 1, mid2 - 1]

Slučaj 4: $\text{key} > \text{arr}[\text{mid2}]$, traženi element je u trećoj trećini niza, ažurira se prostor pretrage u [mid2 + 1, high]

4. Ponavljanje dok se ne isprazni prostor pretrage

Proces se rekursivno ili iterativno ponavlja dok se prostor pretraživanja ne smanji na jedan element.

Ako low prevazilazi high, to znači da u nizu ne postoji traženi element, pa će biti **return -1**.

$\text{arr}[] = \{2, 4, 7, 10, 15, 19, 23, 29, 35\}$ traži se **19**

1. Prostor pretrage $\rightarrow$ [low = 0, high = 8]

Srednje tačke:

$$\text{mid1} = 0 + (8 - 0) / 3 = 2$$

$$\text{mid2} = 8 - (8 - 0) / 3 = 6$$

$$\text{arr}[\text{mid1}] = \text{arr}[2] = 7$$

$$\text{arr}[\text{mid2}] = \text{arr}[6] = 23$$

2. Poređenja:

Pošto je $7 < 19 < 23$, pretraga je u srednjoj trećini
[mid1 + 1, mid2 - 1] = [3, 5]

3. Sledeća iteracija $\rightarrow$ [low = 3, high = 5]

Nove srednje tačke:

$$\text{mid1} = 3 + (5 - 3) / 3 = 3$$

$$\text{mid2} = 5 - (5 - 3) / 3 = 5$$

$$\text{arr}[\text{mid1}] = \text{arr}[3] = 10$$

$$\text{arr}[\text{mid2}] = \text{arr}[5] = 19$$

4. Pronađeno poklapanje

Pošto je $\text{arr}[\text{mid2}] == 19$, vraća index 5.

Na kraju, $\text{key} = 19$ pronađen na indexu 5.

Ternarno pretraživanje - primer

Niz: (9, 7, 1, 2, 3, 6, 10) traži se indeks min. elementa

```
def findMinIndex(arr):  
    low = 0  
    high = len(arr) - 1  
    minIndex = -1  
  
    while low <= high:  
        # podela opsega na 3 dela  
        mid1=low+(high-low) // 3  
        mid2=high-(high-low) // 3  
  
        # ako su mid1 i mid2 jednakih vrednosti  
        if arr[mid1] == arr[mid2]:  
  
            # pretraga je u sredini niza  
            low = mid1 + 1  
            high = mid2 - 1  
  
            # mid1 kao potencijalni minimum  
            minIndex = mid1
```

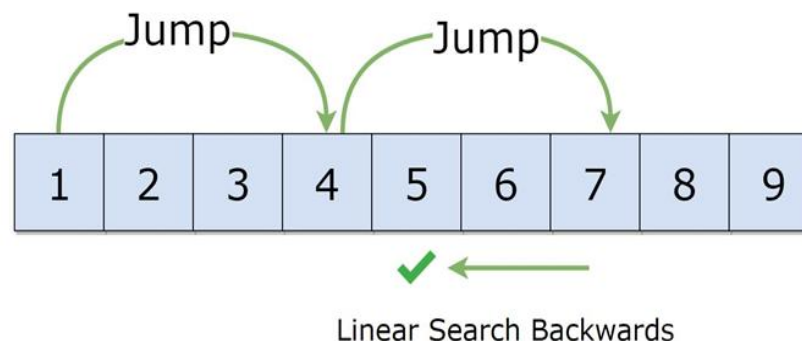
```
# ako arr[mid1] < arr[mid2], minimum je u  
# levoj trećini (uključujući mid1)  
elif arr[mid1] < arr[mid2]:  
    high = mid2 - 1  
    # update sa boljim kandidatom  
    minIndex = mid1  
  
# za arr[mid1] > arr[mid2], minimum je u  
# desnoj trećini (uključujući mid2)  
else:  
    low = mid1 + 1  
    # update sa boljim kandidatom  
    minIndex = mid2  
  
return minIndex  
  
def main():  
    arr = [9, 7, 1, 2, 3, 6, 10]  
    idx = findMinIndex(arr)  
    print(idx)
```

Output

2

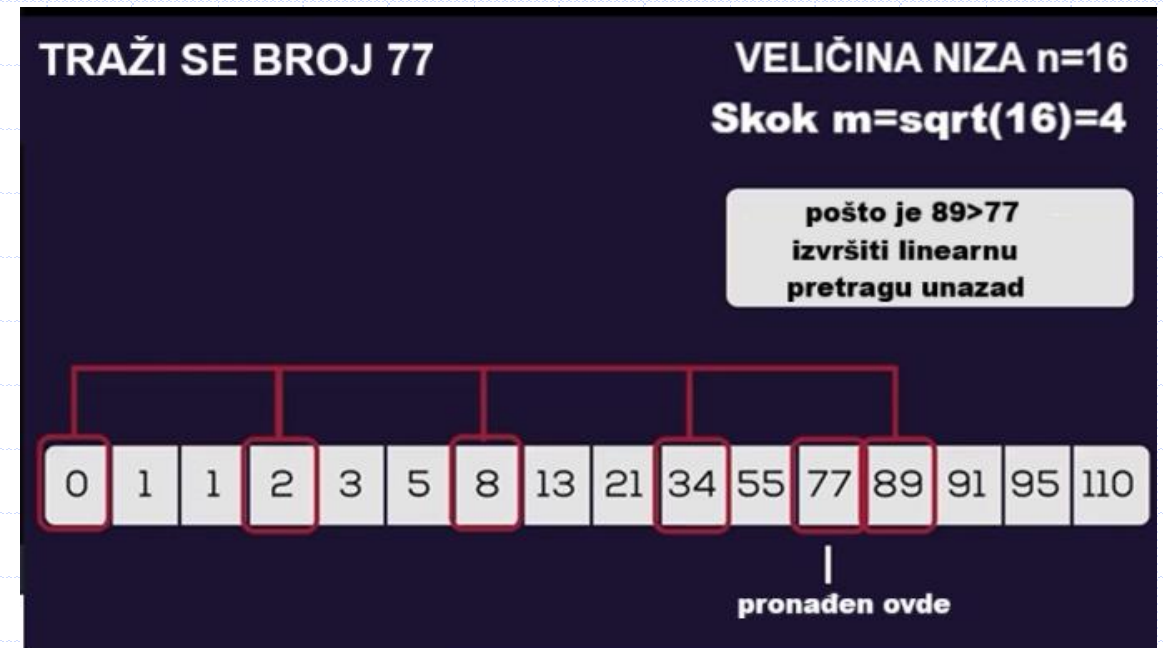
Jump Search pretraživanje

- Kao i kod binarnog pretraživanja, **Jump Search** je algoritam pretraživanja koji se primenjuje na već uređen niz
- Osnovna ideja je da se izvrši provera manjeg broja elemenata niza nego kod linearnog pretraživanja metodom presekavanja pojedinih elemenata niza, umesto pretraživanja svih elemenata
- Ako pretpostavimo da imamo niz $arr[]$ od n elemenata i jump-blok skoka veličine m , tada se vrši pretraživanje elemenata koji su na indeksima $arr[0]$, $arr[m]$, $arr[2m]$,, $arr[km]$ itd.
- Jednom kada se pronade interval $arr[km] < x < arr[(k+1)m]$ izvršava se operacija linearne pretrage od indeksa km kako bi se pronašao element x



Jump Search pretraživanje

- Ako je niz `arr[]` veličine n , a blok preksoka m , za najgori slučaj postojaće n/m skokova, pa će se linearno pretraživanje izvršiti za $m-1$ element niza
- Broj poređenja će biti $n/m+m-1$
- Vremenska složenost algoritma će biti $O(\sqrt{n})$



Jump Search pretraživanje

- Pretraživanje niza: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610]
- Dužina niza je 16. Jump Search pretragom traži se vrednost 55 prema sledećim koracima i veličinom bloka 4:
 - KORAK 1: Skok od indeksa 0 do indeksa 4
 - KORAK 2: Skok od indeksa 4 do indeksa 8
 - KORAK 3: Skok od indeksa 8 do indeksa 12
 - KORAK 4: Pošto je element na indeksu 12 veći od 55, vraćamo se nazad na indeks 8
 - KORAK 5: Izvršiti linearnu pretragu od indeksa 8 da bi se našao broj 55
- Poređenje performansi algoritama pretraživanja:

linear search < jump search < binary search

- Ako je traženi element niza pronađen, vraća se njegov indeks, a ako ne, vraća se -1 što označava da u datom nizu ne postoji traženi element

Jump Search pretraživanje

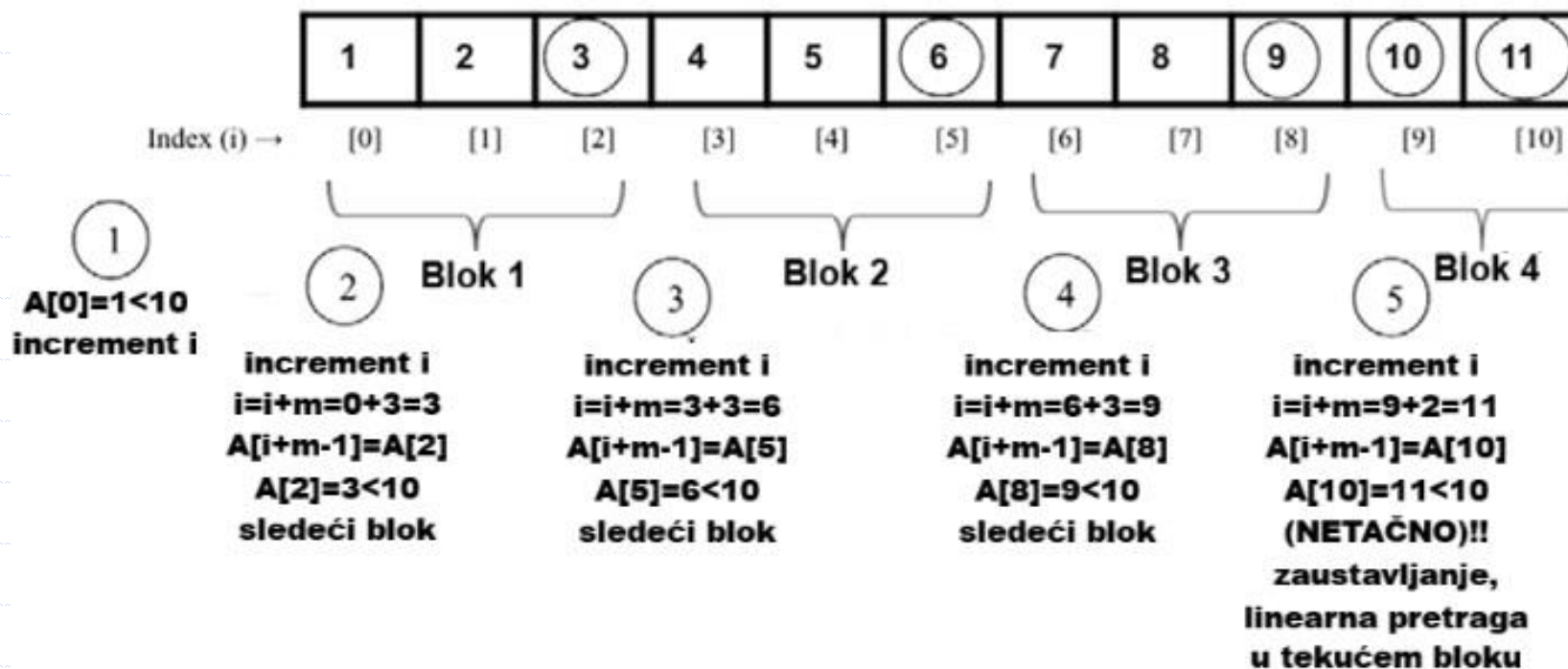
- Kod Jump Search algoritma, prvo se deli sortirani niz podataka na podskupove elemenata koji se nazivaju blokovi
- Unutar svakog bloka, najveću vrednost će imati poslednji element bloka, pošto je niz sortiran
- Zatim se poredi vrednost traženog broja sa poslednjim brojem svakog bloka
- Mogu postojati tri uslova:
 - 1) Ako je traženi broj manji od poslednjeg broja bloka, upoređujemo ga sa poslednjim brojem sledećeg bloka
 - 2) Ako je traženi broj veći od poslednjeg broja bloka, to znači da željena vrednost pretrage mora biti prisutna u trenutnom bloku. Tako da se primenjuje linearna pretraga u ovom bloku i vraćamo poziciju indeksa.
 - 3) Ako je vrednost pretrage ista kao i upoređeni element bloka, vraćamo poziciju indeksa elementa i vraćamo kandidata
- Uopšteno, veličina bloka se uzima kao $\sqrt{n}$, jer daje najbolje performanse za dati niz dužine n

Jump Search pretraživanje - primer

- **Primer:** Niz [1,2,3,4,5,6,7,8,9,10] potrebno je pronaći broj **10**
- Dužina niza je 10, veličina bloka je $m=3$, postoje 4 bloka
- Traži se broj 10 u ukupno 5 poređenja

veličina bloka $m=3$

traži se broj 10



→Prvo se poredi prvi element niza sa traženim brojem 10, pošto je manji, indeks se povećava za veličinu bloka (korak 1)

→Zatim se poredi traženi broj sa poslednjim elementom svakog sledećeg bloka

→Ako je traženi broj veći, prelazi se na sledeći blok, prolazi se kroz blokove 1, 2 i 3

→U bloku 4 je traženi broj manji od poslednjeg elementa 11, zaustavlja se inkrement indeksa, vrši se linearna pretraga u tekućem bloku

Jump Search pretraživanje - kôd

Niz (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610) i traži se vrednost **55**

```
import math

def jumpSearch(arr,x,n):
    # Veličina bloka koji se preskače
    step = math.sqrt(n)

    # Naći blok gde je element x (ako postoji)
    prev = 0
    while arr[int(min(step, n)-1)] < x:
        prev = step
        step += math.sqrt(n)
        if prev >= n:
            return -1

    # Linearna pretraga x u bloku koji počinje sa prev.
    while arr[int(prev)] < x:
        prev += 1
```

```
# Ako smo stigli do sledećeg bloka ili kraja
# niza, element ne postoji
if prev == min(step, n):
    return -1

# Ako je element x pronađen
if arr[int(prev)] == x:
    return prev

return -1
```

Driver code za testiranje

```
arr = [ 0, 1, 1, 2, 3, 5, 8, 13, 21,
        34, 55, 89, 144, 233, 377, 610 ]
```

```
x = 55
```

```
n = len(arr)
```

Pronaći indeks od 'x' preko Jump Search

```
index = jumpSearch(arr, x, n)
```

Print indeksa elementa 'x'

```
print("Number" , x, "is at index" , "%0f"%index)
```

Output:

Number 55 is at index 10

Jump Search pretraživanje

- Prednosti Jump Search pretraživanja:
 - Efikasnije od linearnog pretraživanja za nizove kod kojih su elementi ravnomerno raspoređeni
 - Ima manju vremensku složenost u poređenju sa linearnim pretraživanjem za velike nizove
 - Broj koraka u Jump Search pretraživanju je proporcionalan kvadratnom korenu veličine niza, što ga čini efikasnijim za velike nizove
 - Jednostavnije za implementaciju u poređenju sa binarnim ili ternarnim pretraživanjem
- Nedostaci Jump Search pretraživanja:
 - Primjenjuje se samo kod sortiranih nizova
 - Optimalna veličina bloka preskoka je $\sqrt{n}$ pa je Jump Search vremenska složenost $O(\sqrt{n})$
 - Vremenska složenost Jump Search je između linearne $O(n)$ i binarne pretrage $O(\log n)$
 - Binarno pretraživanje je efikasnije od Jump Search, ali Jump Search ima prednost što pretragu vraćamo samo jednom (binarna pretraga može zahtevati do $O(\log n)$ skokova)
 - Kod sistema gde je implementacija binarnog pretraživanja kompleksna i skupa, treba uzeti u razmatranje Jump Search pretraživanje

Eksponencijalno pretraživanje

- Eksponencijalno pretraživanje je još jedan algoritam pretraživanja koji se uglavnom koristi kada imamo veliki broj elemenata u nizu ili listi
- Ovo pretraživanje je poznato kao „galopirajuće” ili pretraživanje udvostručavanjem
- Eksponencijalni algoritam pretraživanja se izvršava u sledeća dva koraka:
 - 1) Kada je dat sortirani niz od n elemenata, prvo određujemo podopseg u originalnoj listi gde željeni element pretrage može biti prisutan
 - 2) Zatim koristimo binarni algoritam pretrage da bismo pronašli vrednost pretrage unutar podopsega elemenata podataka identifikovanih u koraku 1
- Da bismo pronašli podopseg elemenata niza, počinjemo da tražimo element u datom sortiranom nizu preskačući 2^i elemenata u svakoj iteraciji, i je indeks elementa niza
- Nakon svakog skoka, proveravamo da li je element pretrage prisutan između poslednjeg skoka i trenutnog skoka
- Ako je element pretrage prisutan, onda koristimo binarni algoritam pretrage unutar ovog podniza, a ako nije prisutan, pomeramo indeks na sledeću lokaciju

Eksplozivno pretraživanje

Sortirani niz sa elementima:

$A=[3, 5, 8, 10, 15, 26, 35, 45, 56, 80, 120, 125, 138]$, traži se element **125**

- Počinje se poređenjem prvog elementa niza ($i=0$, tj. $A[0]$) sa 125, pošto je $A[0] < 125$, skače se na lokaciju 2^i sa $i=0$

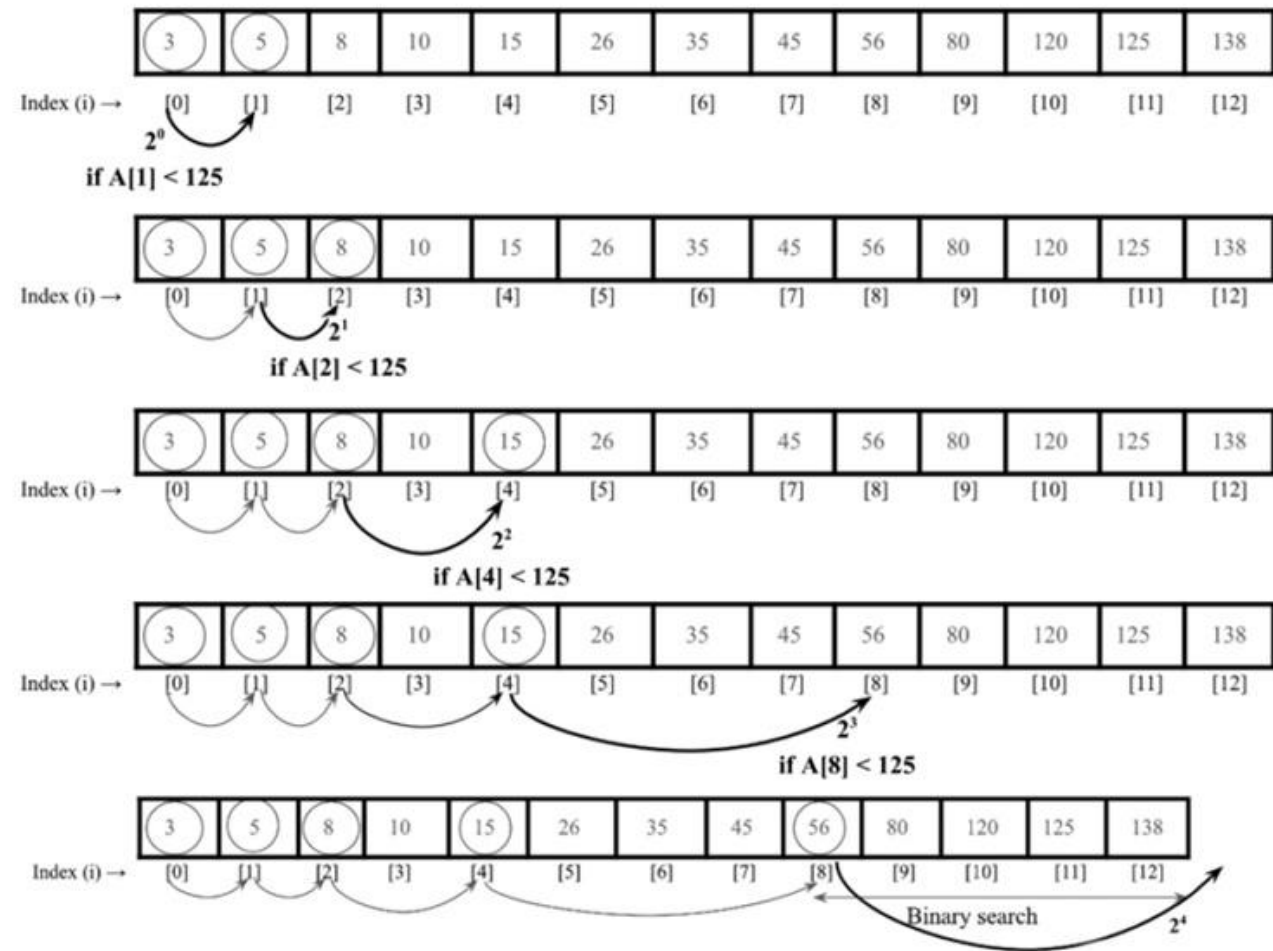
- Pošto je $A[2^0] < 125$, uslov je **true**, skače se na sledeću lokaciju sa $i=1$ tj. $A[2^1] < 125$

- Ponovo se skače na sledeću lokaciju 2^i sa $i=2$, pošto je $A[2^2] < 125$, uslov je true.

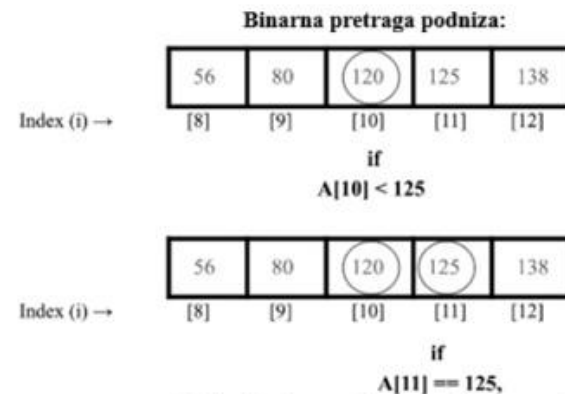
- Iterativno skačemo na sledeću lokaciju sve dok ne završimo pretragu liste ili dok tražena vrednost ne bude veća od vrednosti na tekućoj lokaciji u listi tj.

$A[2^i] < \text{len}(A)$ ili $A[2^i] \leq \text{tražena_vrednost}$.

- Zatim se primenjuje algoritam binarne pretrage u dobijenom podnizu



Ako je $A[16] < \text{len}(A)$ ili
 $A[16] < 125$
uslov je FALSE
zaustavljanje...



Traženi broj se poklapa, prekida se pretraga

Eksponencijalno pretraživanje (rekurzivno) - primer

Niz [2, 3, 4, 10, 40] i traži se vrednost 10

```
# Rekurzivna binarna pretraga vraća lokaciju x u datom
# nizu arr[l..r] ako postoji, ako ne, vraća -1
def binarySearch(arr, l, r, x):
    if r >= l:
        mid = l + ( r-l ) // 2

        # Ako je traženi element = mid
        if arr[mid] == x:
            return mid

        # Ako je element < mid, može biti u levom podnizu
        if arr[mid] > x:
            return binarySearch(arr, l,
                                mid - 1, x)

        # U suprotnom, element je u desnom podnizu
        return binarySearch(arr, mid + 1, r, x)

    # Ako element ne postoji u nizu
    return -1
```

```
# Vraća poziciju prve pojave x u nizu
def exponentialSearch(arr, n, x):
    # Ako je x na prvoj lokaciji u nizu
    if arr[0] == x:
        return 0

    # Opseg za binarnu pretragu (udvostručavanje skoka)
    i = 1
    while i < n and arr[i] <= x:
        i = i * 2

    # Poziv binarne pretrage za pronađeni opseg
    return binarySearch(arr, i//2,
                        min(i, n-1), x)

# Driver Code
arr = [2, 3, 4, 10, 40]
n = len(arr)
x = 10
result = exponentialSearch(arr, n, x)
if result == -1:
    print ("Element not found in the array")
else:
    print ("Element is present at index %d" %(result))
```

Output

Element is present at index 3

Agoritmi pretraživanja - poređenje

Algoritam	Vremenska složenost	Prostorna složenost	Zahtevi
Linearno pretraživanje	$O(n)$	$O(1)$	Bilo koji podaci
Binarno pretraživanje	$O(\log n)$	$O(1)$	Sortirani niz
Ternarno pretraživanje	$O(\log_3 n)$	$O(1)$	Unimodalni podaci
Jump Search	$O(\sqrt{n})$	$O(1)$	Sortirani podaci
Interpolaciono pretraživanje	$O(n)$	$O(1)$	Uniformni podaci
Fibonačijevo pretraživanje	$O(\log n)$	$O(1)$	Sortirani podaci
Eksponencijalno pretraživanje	$O(\log n)$	$O(1)$	Sortirani podaci

